26T1 Week09

Architectural Characteristics & Styles

Software Design & Architecture

Annelise

agenda

- Architectural Characteristics
- Architectural Styles
- Layered Monolith, Modular Monolith, Microservices, EDA

- Lab attendance is mandatory to complete lab09. Nothing to submit; your grade is largely based on your level of engagement in the activity.
- Exam environment sample exam next week
- ass2 part 1 is over!
- ass2 part 2 is due next Wednesday at 3pm

some admin

architectural characteristics

The fundamental high-level structure of a software system. Defines how system components are structured, how they interact and the guiding characteristics that shape its evolution.

Specifically, it focuses on:

- How the **system is divided** into modules, services, or layers for the frontend, backend, database, etc.
- How **data flows** between different components
- What **technologies, frameworks or infrastructure** are used
- **Architectural characteristics** that should be addressed (e.g. security, scalability, performance, etc.)

- AKA non-functional requirements
- Define fundamental qualities software architecture must support
- Influence critical design decisions regarding **structure, behaviour, and trade-offs** in software design
- Impacts the system's **long-term success**, not just initial delivery

architectural characteristics

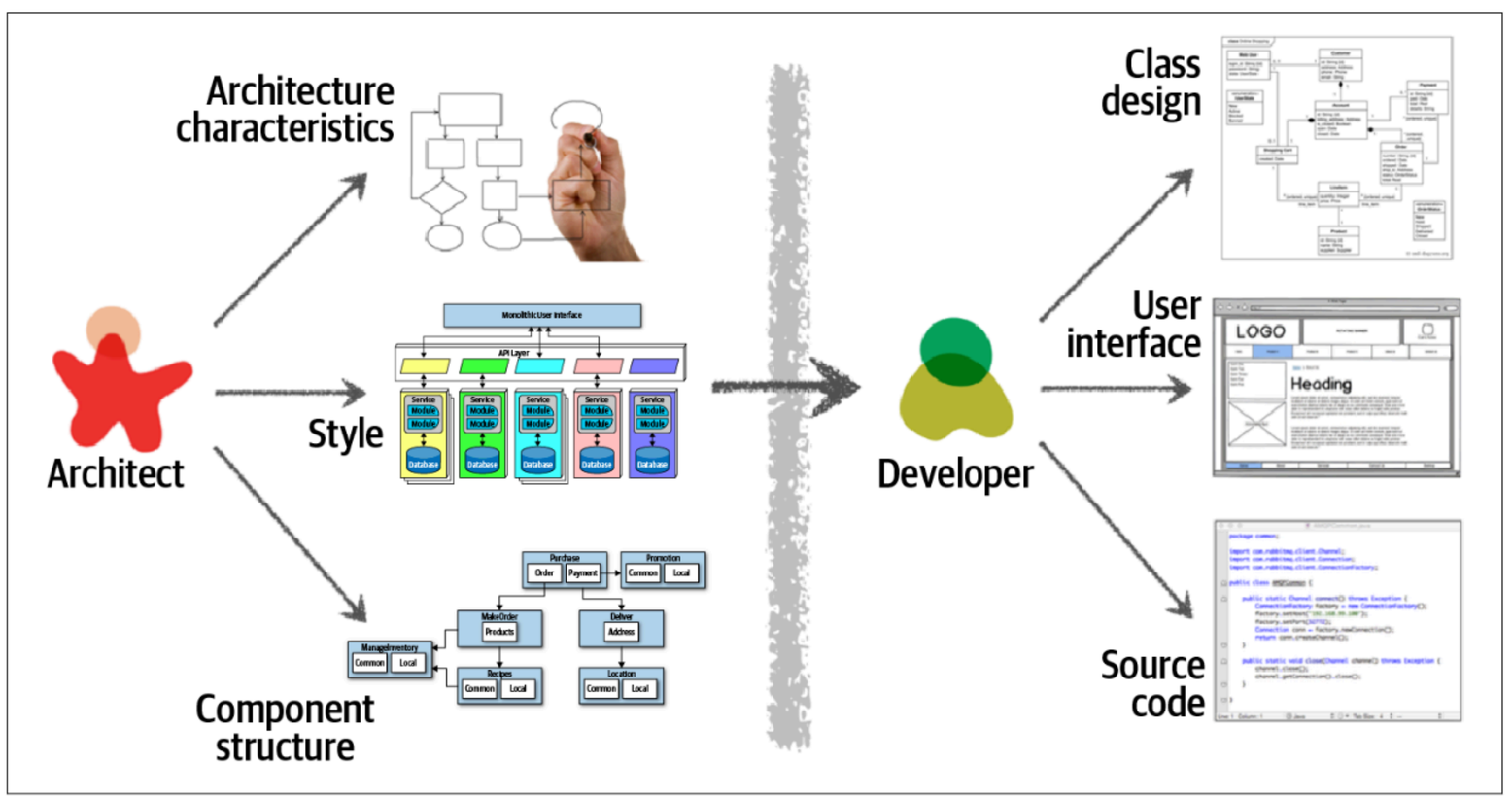
learn your architectural characteristics!

Match each architectural characteristic with its definition or example outcome.

https://www.educaplay.com/learning-resources/26531280-cloud_system_qualities_match.html

Elasticity	Dynamically adjusting computing resources to meet demand
Deployability	Quickly and reliably deploying, with fast rollback capabilities
Availability	Percentage of functional uptime (even with minor failures)
Maintainability	Ease of understanding, repairing, updating, extending the system over time.
Responsiveness	App stays quick and snappy under heavy load
Scalability	System handles more users smoothly by adding resources (CPUs, RAM)
Security	Attempts to hack or steal data are blocked
Modifiability	Code is easy to extend and change as design needs evolve

✦ architectural characteristics ✦



So far, we've been working with concrete code with clear output and tests that tell us if we're right or wrong.

Architecture is a lot less explicit by design! Focus on components, boundaries, and interactions, not code-level details

✦ architecture vs. programming

architectural styles

What are they? + Examples

architectural styles

*Predefined **patterns and philosophies** guiding how software systems are **structured and deployed***

Each style comes with its tradeoffs! Choosing the right style based on the requirements is a core part of software architecture

MONOLITH

Think of...an all-purpose department store

- Everything in one building
- Hard to scale as it grows
- Single point of failure

MODULAR MONOLITH

Think of...a shopping mall

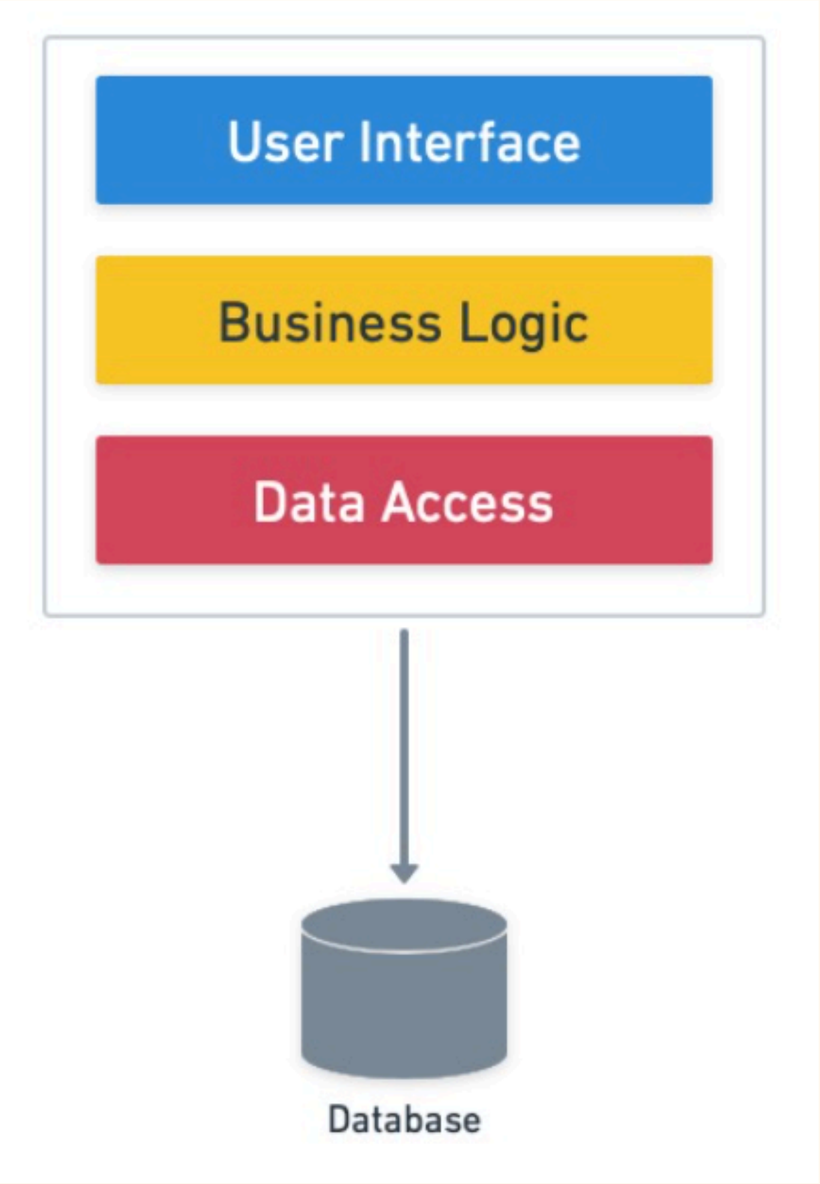
- One building (system), but separated stores (modules)
- Shared infrastructure (parking, elevators, services)

DISTRIBUTED SYSTEMS

Think of...a shopping district

- Separate buildings
- Independent shops with their own systems
- More scalable, more complex coordination

✦ monoliths vs. distributed systems ✦



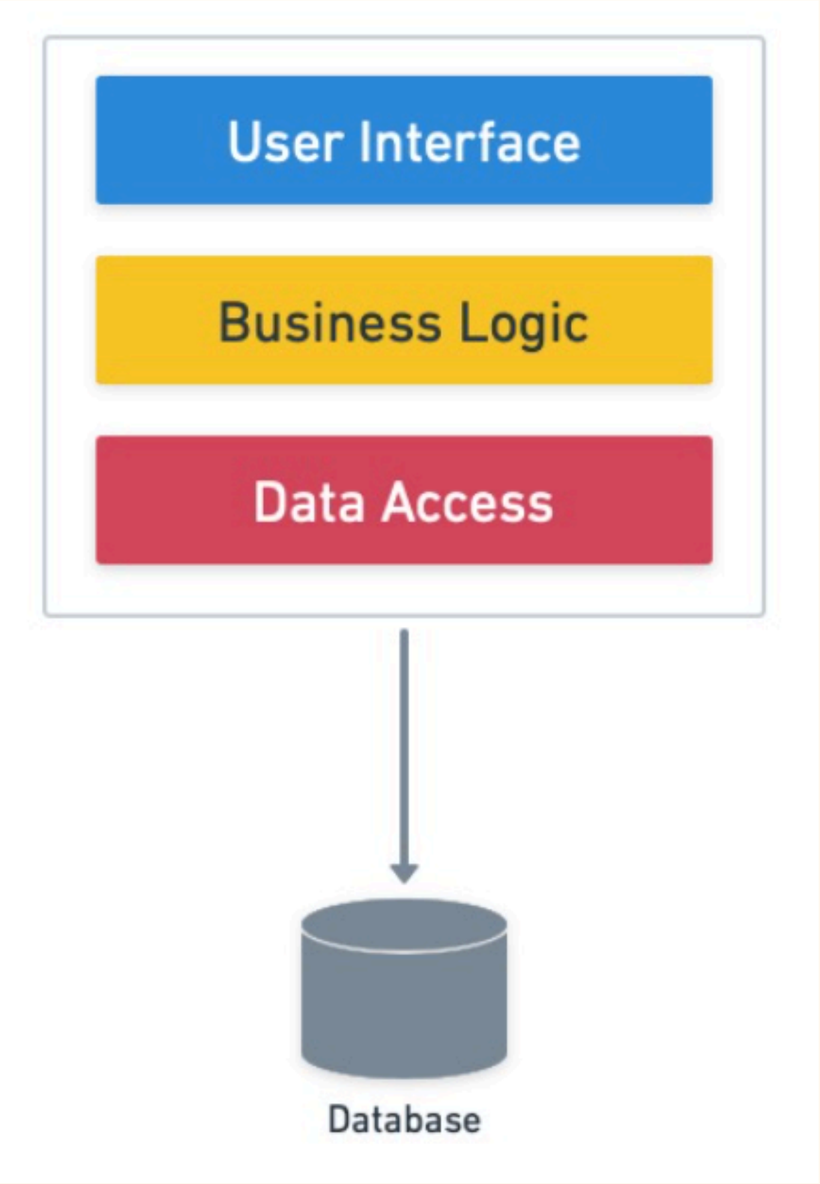
Organises each distinct technical responsibility into separate layers.

Domain logic spans multiple layers. PWP

- Presentation (UI components)
- Workflow (business logic components)
- Persistence (database schemas and operations)

Easy to understand and lets you build simple systems fast

layered monolith



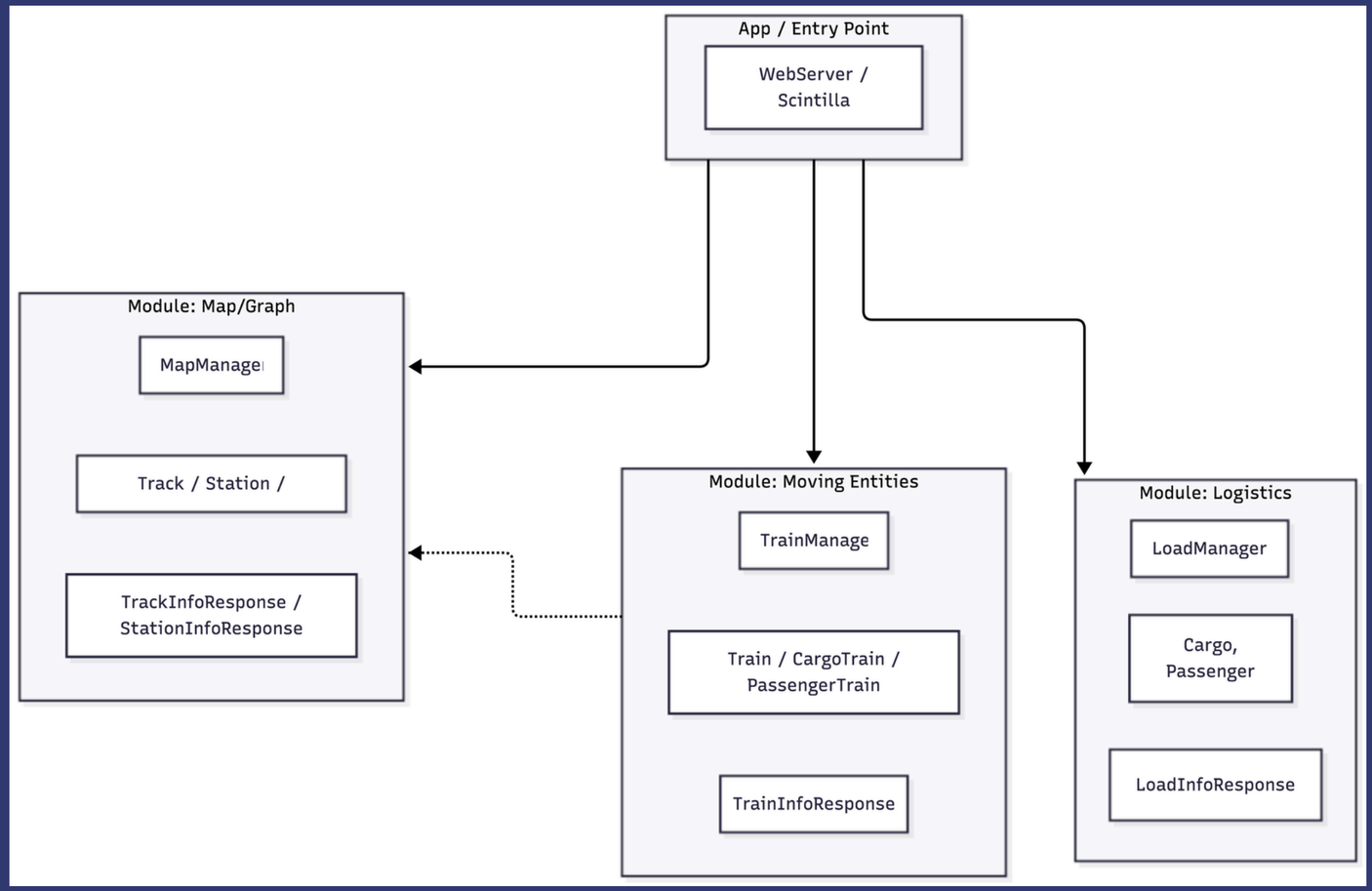
Strengths

- **Simple, cost-effective** structure with **centralised data**
- **Strong performance** due to in-process communication

Challenges

- Highly **coupled**
- **Hard to scale, test, or deploy independently** as the system grows

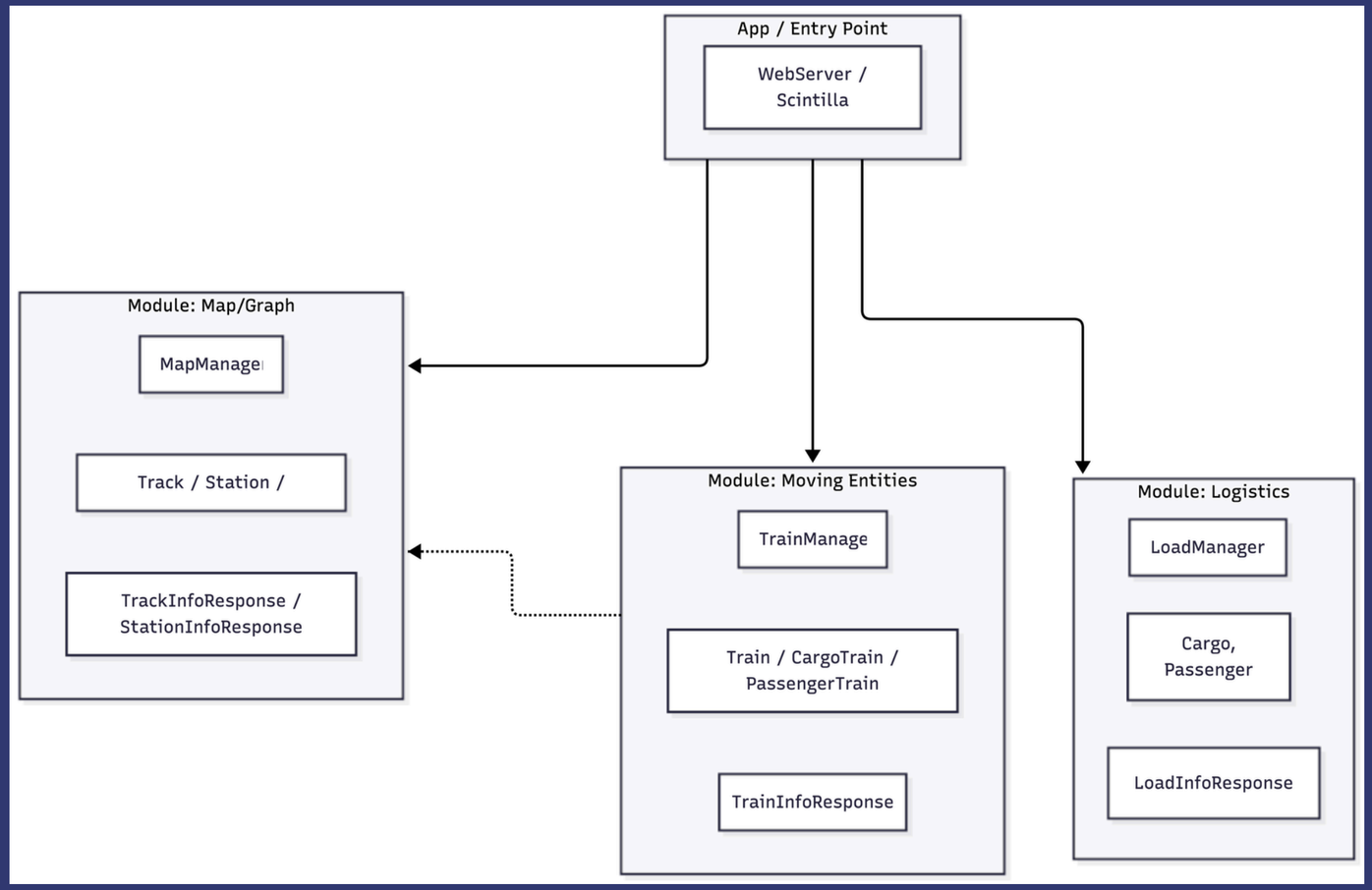
layered monolith



Deployed as a single unit and organised in domain-based modular structure

- Each domain is represented as a module
- Shared database
- Different parts of the app communicate through public controller interfaces

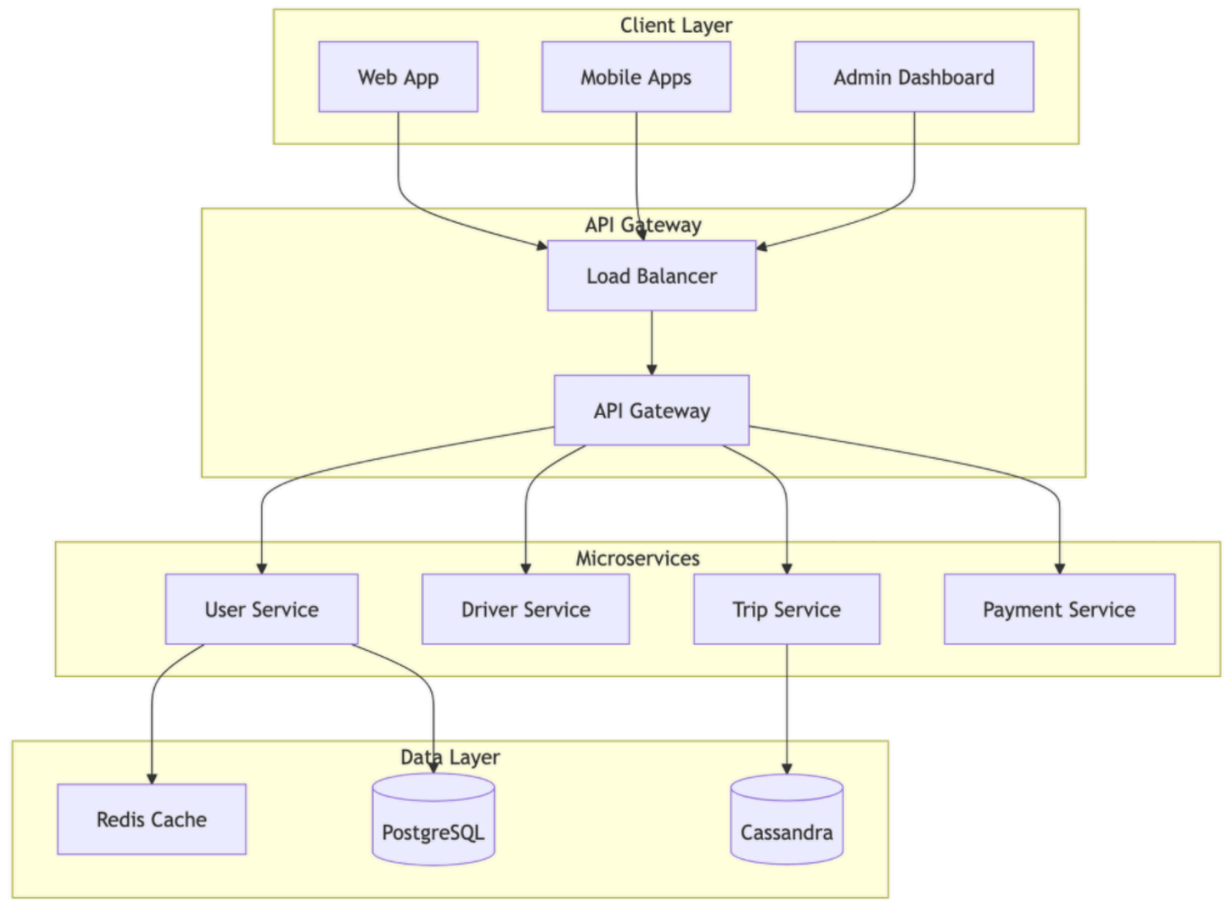
modular monolith ✸



Strengths: Better maintainability and testability through domain-based modules while retaining monolith performance and simple deployment

Challenges: Limited scalability and fragile modular boundaries that become harder to maintain as teams and codebase grow

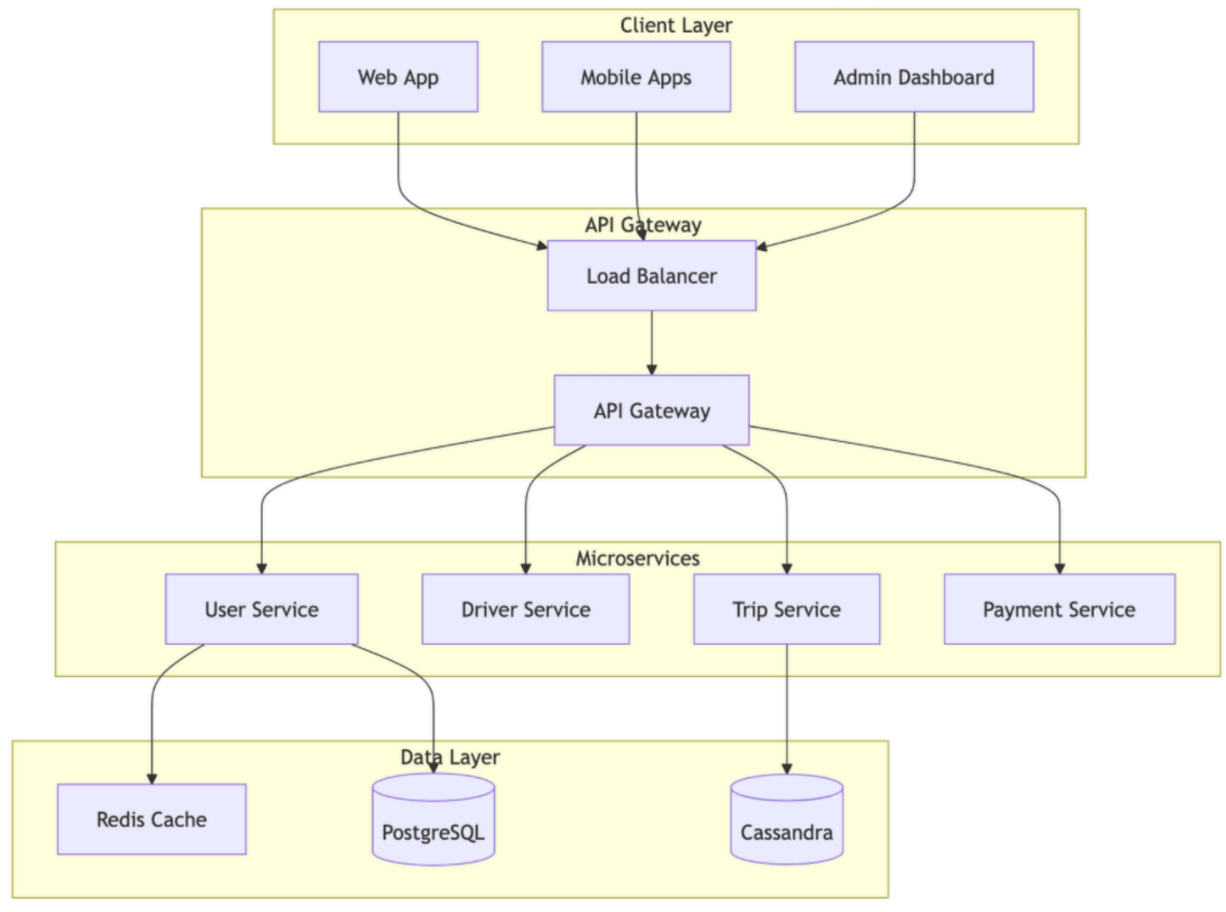
modular monolith ✸



microservices ★

Single-purpose, separately deployed units for environments requiring **frequent changes and scalability**

- A microservice performs one specific function very well and communicates with other microservices over a **network**
- Each service **owns their own data**. Direct access to data is restricted to the owning microservice
- Balance of granularity disintegrators (smaller services) and integrators (larger services)



Strengths: High **scalability**, **independent deployment**, and strong **fault isolation** with smaller, focused services

Challenges: Increased system **complexity** with **difficult debugging**, **network overhead**, and distributed **data management**

microservices ★

event-driven architecture

Structures systems to respond to events, which are significant changes in a system state.

- Components in a system communicate faster by producing and responding to events asynchronously
- Events broadcast something that has already happened across the entire system and do not have responses (async)
- EDA can choose between monolithic, domain-partitioned or database-per-service databases

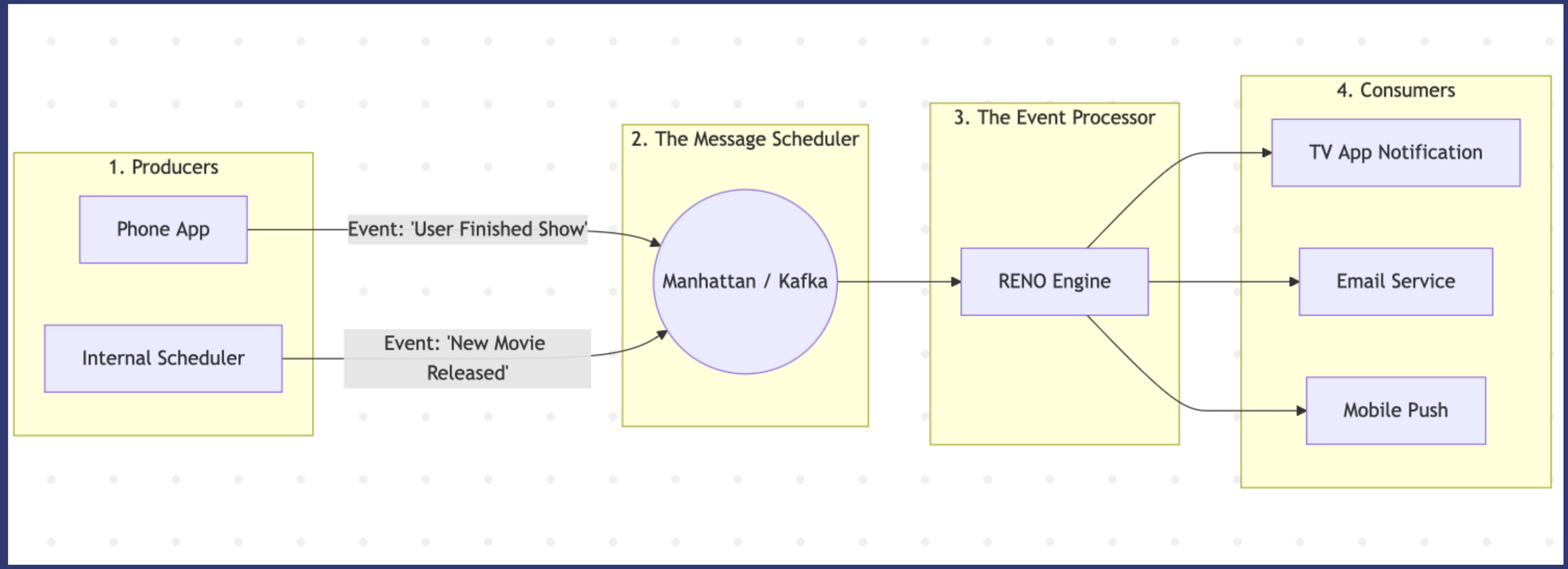
event-driven architecture

* * *

Strengths: Highly **decoupled**, **scalable**, and **fault-tolerant** with flexible, asynchronous processing

Challenges: **Hard to test and debug** due to asynchronous flows; not suitable for tightly coupled, synchronous workflows

event-driven architecture



case study.

How do requirements actually shape architecture?

✦ ✦ ✦

Architectural requirements drive changes all the time.

GitLab and Atlassian (Confluence) both started as monoliths, but their goals were different. This led them down different paths.

different requirements

GITLAB'S GOALS

- Boost development speed and predictability
- Improve code quality
- Reduce coupling
- Enable independent deployment of components

Key qualities
Maintainability, modifiability, deployability

Source: Gitlab's ADR

ATLASSIAN'S GOALS

- Scale globally
- Improve performance
- Ensure resilience
- Give teams autonomy

Key qualities
Scalability, performance, reliability, team autonomy

Source: Atlassian's Engineering Blog

different solutions

GITLAB
→ ***MODULAR MONOLITH***

- Kept one app but organised code into clear modules
- Reduces coupling, adds useful abstractions
- Lets parts of the app be deployed separately if needed

ATLASSIAN
→ ***STATELESS MONOLITH***
→ ***MICROSERVICES***

- Made the app stateless and multi-tenant for global scale
- Gradually split into microservices for resilience & team autonomy
- Routing layer allows smooth migration without breaking anything

discuss.

Is it better to optimise early for scalability or wait until it becomes a problem?

design for scalability, optimise on demand. ✦

Design for Scalability Early

- Keep scalability in mind from the outset.
- Select architectural patterns, technologies, and data storage solutions that are inherently capable of scaling.
- Changing fundamental architectural choices later to accommodate scalability is often extremely difficult and costly.

Optimise on Demand

- Full optimisation for extreme scalability from day one can be wasteful
- Instead, identify bottlenecks through monitoring and testing as the system grows, and then apply specific optimisations to those problematic areas.
- Saves resources and allows focus on immediate needs

discuss.

Can a modular monolith achieve the same team autonomy as microservices?

not the same, but an improvement ✸

Modular Monolith

- Teams can work independently on specific modules; autonomy over code within that module, reduces conflicts.
- But still a shared deployment pipeline, potentially a single release schedule, and constraints on the shared tech stack and overall build process.
- Coordination for major releases or infrastructure changes is still necessary

Microservices

- Independent deployment units, owned end-to-end by a single team; true operational and technological autonomy.
- Teams can deploy on their own schedule, choose their own tech stack (within reason), and manage their own infrastructure

discuss.

Are there systems where event-driven architecture causes more harm than good? What would that look like?

yes, EDA sometimes does more harm than good. ✴

1. **Systems Requiring Strong, Immediate Consistency** across components
- e.g. Core banking ledgers, critical inventory management where simultaneous updates must be globally consistent immediately
- Managing eventual consistency, distributed transactions, and potential compensating actions can become overly complex

2. **Simple, Small Applications with Limited Complexity** with minimal integration needs and low traffic
- Sometimes the operational overhead and debugging complexity of an EDA isn’t helpful
- A simpler approach would be more efficient and easier to manage (e.g. monolithic)

discuss.

Should every modern application aim to use Microservices?

no ✦

Appropriate Use Cases

- Generally well-suited for **large, complex, evolving systems** that require **independent scalability** of components, **technology diversity**, and autonomous teams

Drawbacks for Other Cases

- Overhead of managing a distributed system can far outweigh the benefits
- A well-designed monolith or modular monolith can often be a more pragmatic and efficient choice
- Small, simple applications, startups with limited resources, or applications with stable requirements
- Choose based on genuine business needs and organisational capabilities

discuss.

How do you decide whether to prioritise maintainability or performance in an architecture?

PRIORITISE PERFORMANCE WHEN...

- **Real-time response** is critical e.g., high-frequency trading, autonomous vehicle control, interactive gaming.
- **High throughput** is essential e.g., advertising platforms, analytics systems processing vast data streams.
- **User experience is directly tied to speed** e.g., e-commerce checkouts, search engines where even slight delays cause user abandonment.
- **Resource efficiency is a primary cost driver**: Minimising server costs through highly optimised code.
- Example: For a financial trading system, a millisecond delay could mean millions of dollars lost. Performance would heavily outweigh some maintainability compromises.

consider primary business drivers, user expectations, and long-term strategic goals.

PRIORITISE MAINTAINABILITY WHEN...

- **System longevity and evolution** are key: Most enterprise applications undergo continuous updates and feature additions over many years.
- **Cost of change** needs to be low
- **Team onboarding** and productivity are important
- System requirements are expected to **change/adapt** frequently
- Example: A complex internal ERP system will likely be maintained for decades.
    - Easily adding modules or fixing bugs without breaking existing functionality would be far more valuable than shaving off a few milliseconds of response time for non- critical operations.

consider primary business drivers, user expectations, and long-term strategic goals.

lab time!

SitarME306